

Automatisierung der PSV-Soft / VibSoft via COM

Beispiel: Erstellung einer *.exe zum externen Start eines PSV-Scans

Grundgerüst

Ziel: Erstellung eines Programms, das via COM (Component Object Model) die PSV-Software fernsteuert

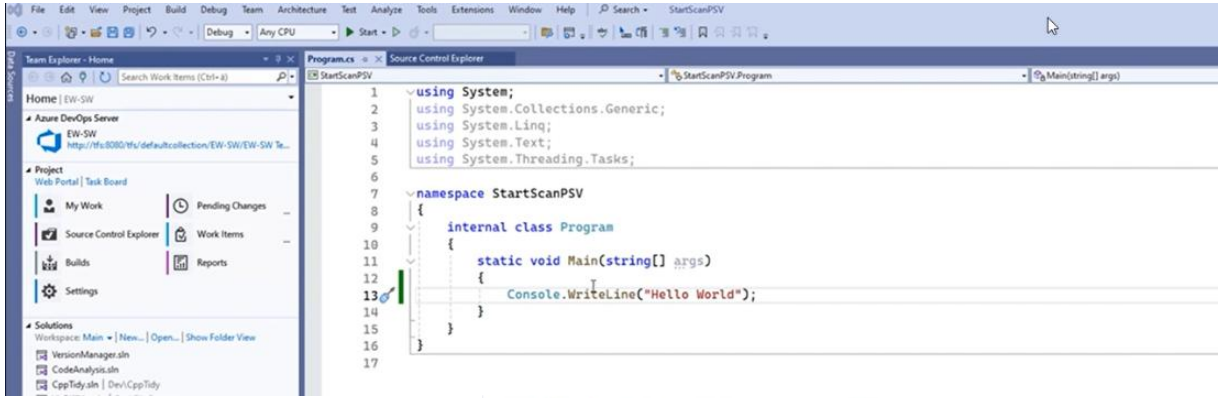
- Start der Entwicklungsumgebung 

- Create new project (C#)
- Konsolenapplikation unter Windows mittels Console App (.NET Framework)
- Projektname (StartScanPSV) erstellen
- Klassiches .NET Framework 4.7.2 (nicht das neue!) auswählen

Anmerkung DAL (04.08.2026):
Auf die COM-Schnittstelle kann auch per Python zugegriffen werden.

Grundgerüst

- Fenster öffnet sich, das man z. B. mit dem Zusatz Console.WriteLine(„Hello World“); ergänzen kann:

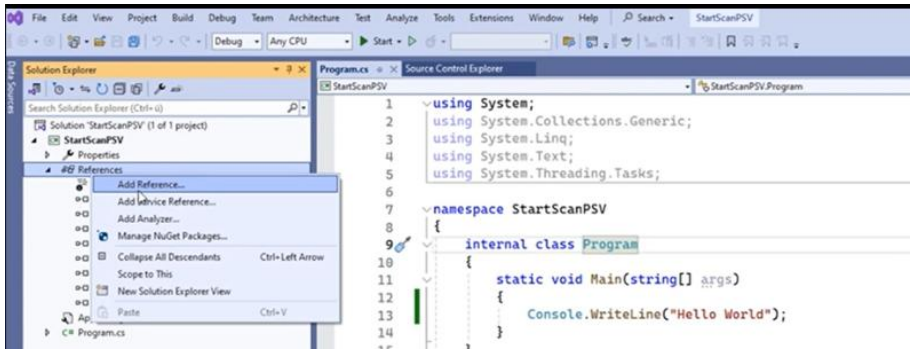


- Ausführung des Programms generiert Ausgabe-Fenster mit „Hello World“

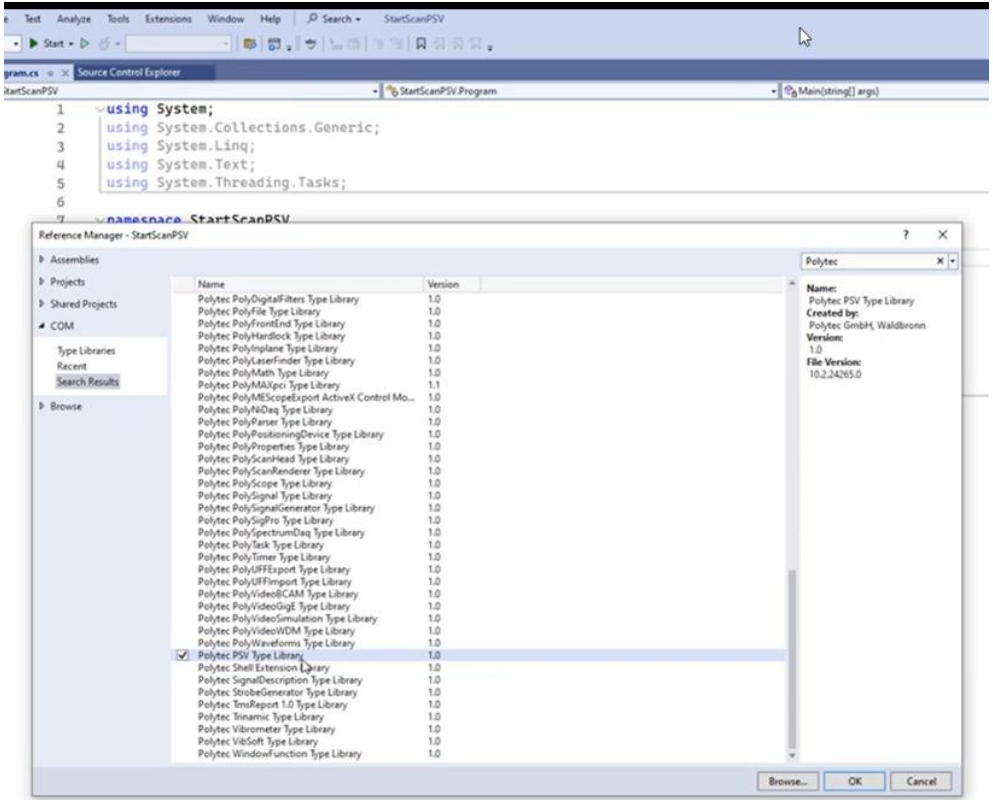


Code erweitern: Generelles

- Um PSV zu steuern, muss eine Reference auf die PSV-Software erstellt werden (PSV muss dazu installiert sein)

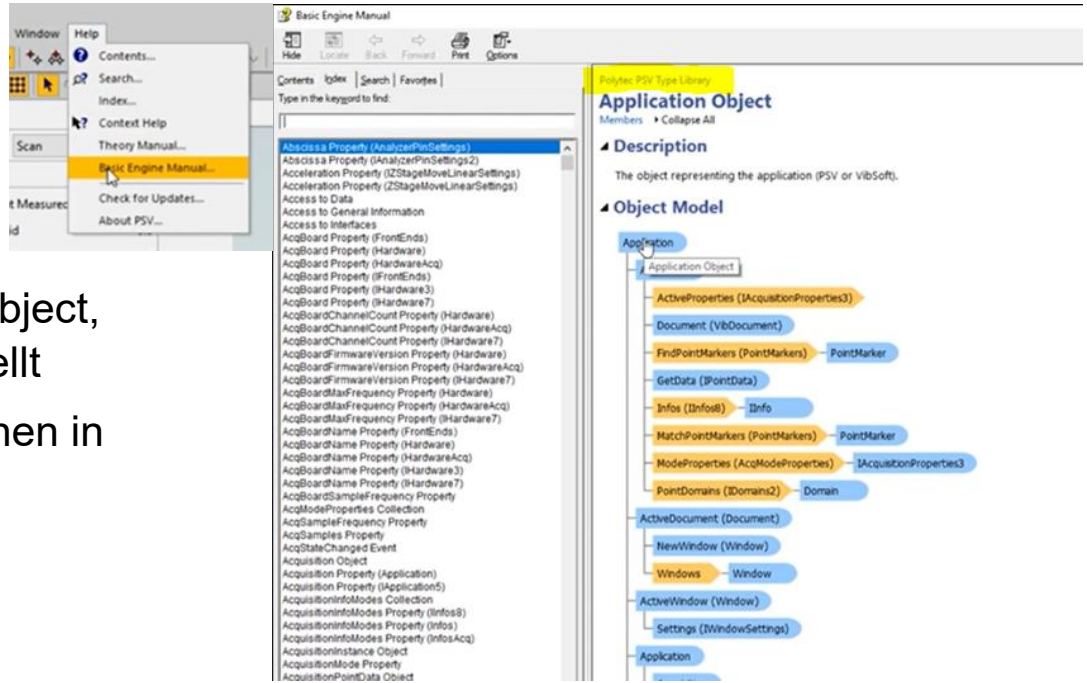


- Unter dem Abschnitt COM findet man viele Typ Libraries, die Softwarekomponenten beinhalten
- Die Polytec PSV Type Library ist die Beschreibung der COM-Objekte, die in der PSV-Software enthalten sind (in der Liste, weil PSV installiert ist)



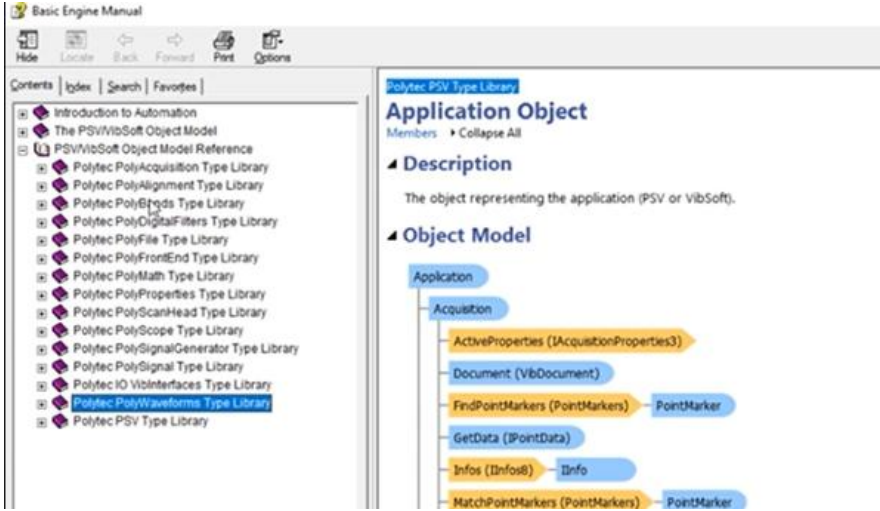
Hilfe erhalten

- Polytec PSV Type Library ist im Basic Engine Manual aufgeführt und erklärt
- Start des Ganzen ist das Application Object, das die komplette PSV-Software darstellt
- Dieses Objekt ist wie ganz oben zu sehen in der Polytec PSV Type Library



Hilfe erhalten

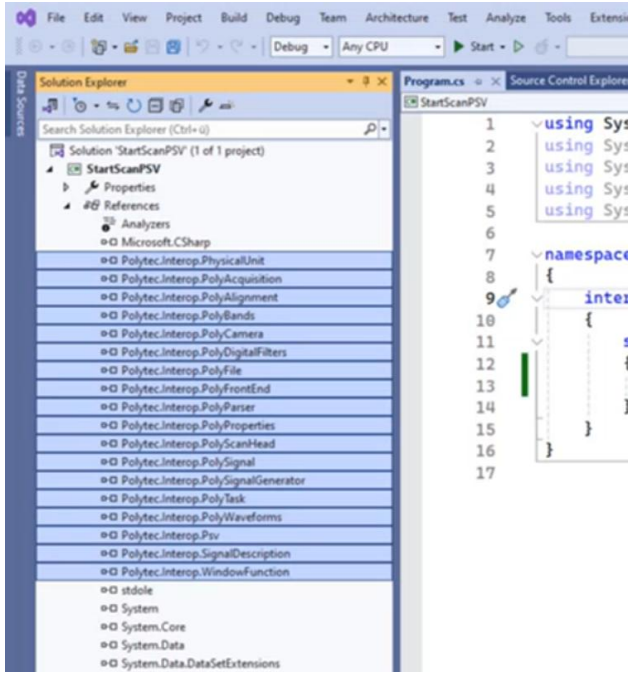
- Unter Contents sieht man, dass das Objektmodell von PSV in mehreren Type Libraries enthalten ist. Die PolyWaveforms Type Library benötigt man beispielsweise zusätzlich, wenn man die Art der Anregung ändern möchte (Chirp statt Sinus)
- Zum bloßen Aktivieren des Generators kann jedoch weiterhin die PSV Type Library genutzt werden mit der GeneratorsOn Property





übrigens

- Die Entwicklungsumgebung erkennt, dass in PSV Type Library auch andere Libraries mitbenutzt werden und zeigt diese deshalb direkt mit an
- Zugriff ist somit auf das komplette Objektmodell möglich
  (auch auf PolyWaveforms)



Code erweitern: Konkret für Scan-Programm

- Da eine eigenständige *.exe erstellt werden soll, die parallel zu bzw. außerhalb von PSV läuft, gibt es das Objekt PresentationInstance, das eine Verbindung zur Auswertung herstellt
- Für die Datenerfassung muss hingegen das Objekt AcquisitionInstance angelegt werden
- Dieses wird nach der Applikation befragt und dann bekommt man das Application-Objekt angezeigt, dass zur Datenerfassung der PSV-Software gehört.
- Die Hilfe zeigt den Code als Visual Basic, der anschließend in C# überführt werden muss



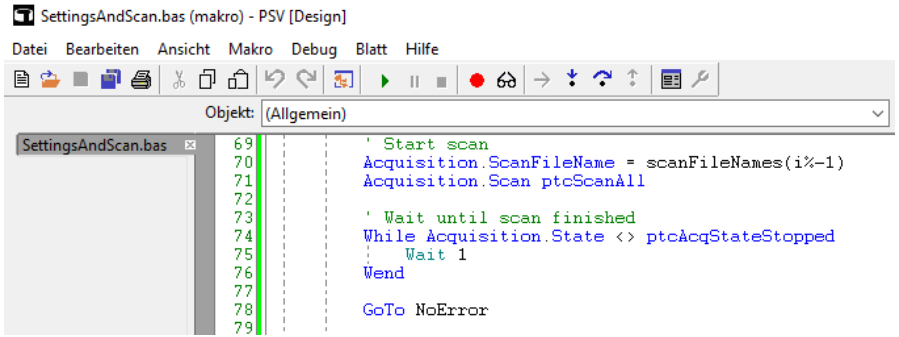


- Es wird ein neues AcquisitionInstance-Objekt angelegt und der Variable acquisitionInstance zugewiesen;
- Mit der nächsten Zeile wird ein Timeout von 10000ms angegeben, damit die PSV-Software 10s Zeit zum Starten hat
- Danach gibt das Programm die PSV-Versionsnummer aus

```
7   namespace StartScanPSV
8   {
9       internal class Program
10      {
11          static void Main(string[] args)
12          {
13              Console.WriteLine("Hello World");
14
15              var acquisitionInstance = new Polytec.Interop.Psv.AcquisitionInstance();
16              var psv = acquisitionInstance.GetApplication(true, 10000);
17              Console.WriteLine(psv.Version);
18          }
19      }
```

Hilfe aus bestehenden Makros

- Nun soll es erweitert werden, damit ein Scan gestartet werden kann
- Hierzu kann man im bestehenden VBA-Makro SettingsandScan.bas einen Befehl ablesen und für C# umschreiben



Programm finalisieren

- Als ersten muss also das Acquisition-Objekt eingebunden werden
- Danach wird ein Pfad definiert, wo der Scan gestartet werden soll
- Abschließend wird der Scan gestartet und so lange mit „while“ gewartet, bis der Scan fertig ist

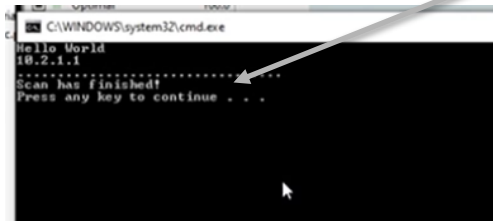
```
static void Main(string[] args)
{
    Console.WriteLine("Hello World");

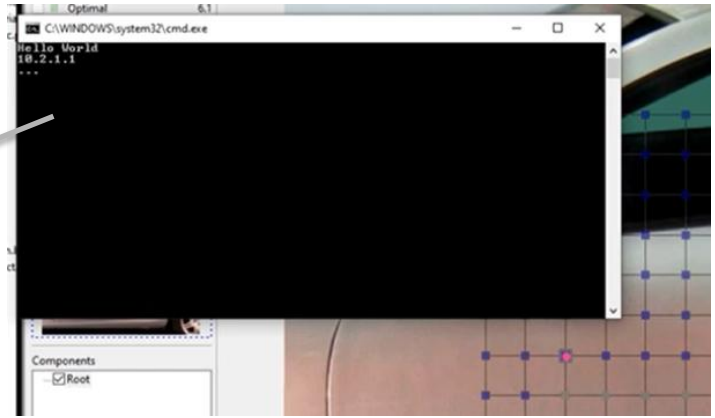
    var acquisitionInstance = new Polytec.Interop.Psv.AcquisitionInstance();
    var psv = acquisitionInstance.GetApplication(true, 10000);
    Console.WriteLine(psv.Version);

    var acquisition = psv.Acquisition;
    acquisition.ScanFileName = @"D:\Temp\Scan.svd";
    acquisition.ScanEx(Polytec.Interop.Psv.PTCScanMode.ptcScanAll, Polytec.Interop.Psv.PTCScanCaps.ptcScanCapsNone);

    while (acquisition.State != Polytec.Interop.Psv.PTCAcqState.ptcAcqStateStopped)
    {
        Console.Write(".");
        // sleep current thread for 1 s
        System.Threading.Thread.Sleep(1000);
    }

    Console.WriteLine("\nScan has finished!");
}
```





übrigens

- Dieses COM-Programm läuft somit lokal auf dem Rechner, auf dem PSV installiert wurde
- Möchte man einen fremden PC steuern benötigt man DCOM
  (Distributed Component Object Model)
  - Hier würde man dann noch den Server bei der AcquistionInstance mit angeben müssen, unter der die andere Instanz von PSV läuft.
  - Vor allem müssen sehr viele aufwendige Sicherheitseinstellungen und Konfigurationen gemacht werden, damit Windows DCOM zwischen beiden Rechnern zulässt!